// HANDOFF PARA A EQUIPE

GUIA DO PROJETO

Arquitetura, convenções, infraestrutura, inscrições e como contribuir.

Versão 0.1.0 · atualizado em 10 de outubro de 2026
https://github.com/Marcos-Codfy/lp-hackarun

Documento de handoff: tudo que uma pessoa nova precisa pra rodar, entender, alterar e publicar a landing page. Mantido junto do código. O PDF é gerado a partir deste arquivo com `npm run docs:pdf`.

1. Links rápidos

Item	Valor
Repositório (público)	https://github.com/Marcos-Codfy/lp-hackarun
Branch de produção	main
Hospedagem	Cloudflare Pages, plano gratuito (https://lp-hackarun.pages.dev depois de conectar)
Inscrições	Formulário do site !’ Google Apps Script (Web App) !’ Google Sheets
Node.js	24 LTS (arquivo .nvmrc), mínimo 22.12
Especificação de design	docs/design/design-spec.md
Decisões de arquitetura	docs/adr/
Stack e padrões (PDF)	docs/pdf/HackaRun_LP_Stack_e_Padroes.pdf

2. O que é

Landing page do **HackaRun 2026**, o hackathon dos cursos de Engenharia de Software e Inteligência Artificial da **Unicatólica**, com parceria do **Capim Dourado Shopping**. Evento de 11 a 14 de novembro de 2026, com minicursos, mentoria de egressos, corrida de missões (os NPCs dão pistas) e premiação no shopping com telão e banda.

Objetivos da página: explicar o evento, mostrar a programação e as missões, listar as equipes confirmadas e **converter em inscrições**.

3. Decisões principais

Decisão	Escolha	Motivo curto
Framework	Astro (site estático) + TypeScript	HTML pronto no build, quase zero JS inicial, ótimo LCP e SEO
Hero	Partículas 3D procedurais (Three.js + shaders)	0 KB de vídeo/modelo, nítido em qualquer tela, interativo e muda com o scroll
Animação	GSAP + ScrollTrigger + SplitText, Lenis	Padrão de mercado pra scroll storytelling, gratuito
Hospedagem	Cloudflare Pages (grátis)	Banda e requisições ilimitadas pra estático, CDN no Brasil, aguenta pico
Inscrições	Google Sheets via Apps Script	Gratuito, a organização já usa Google, planilha vira o painel
Arquitetura	Clean Architecture + DDD + MVVM	Camadas claras, regras testáveis, fácil de dividir entre várias pessoas

Detalhes e alternativas descartadas estão em `docs/adr/`.

4. Como rodar

```
git clone https://github.com/Marcos-Codfy/lp-hackarun.git
cd lp-hackarun
npm ci
cp .env.example .env
npm run dev
```

Abra `http://localhost:4321`. Sem `PUBLIC_REGISTRATION_ENDPOINT` o formulário roda em **modo simulado** no ambiente de desenvolvimento (não grava em lugar nenhum).

Script	O que faz
npm run dev	Servidor local com hot reload
npm run build	astro check + build de produção em dist/
npm run preview	Serve o dist/ localmente
npm run lint / lint:fix	ESLint (inclui as regras de dependência entre camadas)
npm run format	Prettier em tudo
npm test / test:watch	Testes unitários (Vitest)
npm run validate	lint + testes + build (o mesmo que a CI roda)
npm run docs:pdf	Gera os PDFs de documentação

5. Estrutura de pastas

```
src/
  domain/            regras de negócio puras (entities, value-objects, services,
repositories = interfaces)
  application/       casos de uso, DTOs e portas (Clock, CalendarExporter,
RegistrationGateway)
  infrastructure/    adaptadores: conteúdo JSON, repositórios, Apps Script, áudio,
storage, calendário
    content/         >>> conteúdo editável do site (event.json, stations.json,
faq.json...)
  presentation/
    app/             bootstrap do browser (App), contratos de eventos e contexto
    layouts/         BaseLayout (head, fontes, SEO, canvas 3D)
    sections/        uma seção da página por arquivo (Hero, Missões, Programação...)
    components/      ui/, layout/, countdown/, registration/, teams/, hud/, npc/,
sound/, seo/
    view-models/     ViewModels (MVVM): estado + lógica de tela, sem DOM
    views/           Views (MVVM): ligam DOM <-> ViewModel
    motion/          core/ (Lenis, ScrollTrigger, SceneDirector) e animators/ (entradas
das seções)
    experience/      3D: palco three.js, shapes, shaders, qualidade adaptativa
    styles/          tokens.css (design tokens) e base.css
  shared/            utilitários sem regra de negócio (Observable, EventBus, Ticker,
math, dom)
  composition/       composition roots: server.ts (build) e client.ts (browser)
  pages/             rotas do Astro (index, 404, .ics)
apps-script/         código do Google Apps Script (API de inscrições) + passo a passo
tests/               testes unitários espelhando as camadas
docs/                guia, design spec, ADRs, manifesto da stack e PDFs gerados
scripts/docs/        gerador dos PDFs
design/references/   referências visuais e o vídeo teaser (não vão pro site)
public/              arquivos servidos como estão (favicon, _headers, robots, áudio)
```

6. Arquitetura

6.1 Camadas e regra de dependência

As dependências sempre apontam para dentro: `presentation` !`application` !`domain`. `infrastructure` implementa as interfaces (portas) declaradas em `domain`/`application`. Quem junta tudo são os **composition roots** em `src/composition`. O ESLint **bloqueia** imports proibidos (ex.: domínio importando three.js ou infraestrutura).

6.2 Fluxo no build (SSG)

1. `src/pages/index.astro` chama `serverContainer().getLandingPage.execute()`.
2. O caso de uso lê os repositórios estáticos (JSON em `infrastructure/content`), que passam pelos **mappers** e viram

entidades/aggregate. Dado inválido quebra o build (fail fast).

3. A aplicação devolve **DTOs** serializáveis, e as seções renderizam HTML puro.
4. Um JSON (`script#app-config`) leva pro browser só o necessário: marcos do contador e dados da inscrição.

6.3 Fluxo no browser

`App.boot()` monta, em ordem de prioridade:

1. `mountMotion`: Lenis (smooth scroll) + ScrollTrigger + **SceneDirector** (acompanha as seções com `data-scene`).
2. `mountViews`: Views MVVM (contador, som, equipes ao vivo, formulário, HUD, NPCs). Funcionam sem 3D.
3. `mountAnimators`: entradas animadas das seções (GSAP).
4. `mountExperience` (chunk separado, carregado quando o navegador fica ocioso): o 3D.

6.4 MVVM

- **ViewModel** (`presentation/view-models`): estado em `Observable`, chama casos de uso, não toca no DOM. Testável.
- **View** (`presentation/views`): assina o ViewModel, atualiza o DOM e repassa eventos do usuário.
- **Model**: casos de uso + domínio.

6.5 3D guiado por scroll

Cada seção tem `data-scene="<forma>"`. O `SceneDirector` transforma a posição do scroll em um índice contínuo (`scenes.progress`, ex.: 3,4). O `SceneMixer` converte isso em pesos por forma, e o shader mistura as posições das partículas na GPU. Formas: `core`, `nebula`, `enigma`, `route`, `arena`, `screen`, `trophy`, `dust`.

Adicionar uma forma nova: crie uma classe `ShapeStrategy` em `experience/shapes`, registre no `ShapeFactory`, adicione a chave em `SceneKey` (`presentation/app/AppEvents.ts`) e use `data-scene="<chave>"` na seção.

6.6 Eventos entre módulos

Os módulos conversam pelo `EventBus` tipado (`presentation/app/AppEvents.ts`): `scene:enter`, `experience:ready`, `experience:charging`, `experience:charged`, `sound:changed`, `npc:found`, `registration:intent`, `registration:submitted`.

7. Editar conteúdo sem programar

Tudo em `src/infrastructure/content/`. Edite, rode `npm test` (os testes validam o conteúdo real) e abra um PR.

Arquivo	O que controla
event.json	Datas, locais, marcos do contador e inscrição (endpoint, closesAt, capacity, teamSize)
stations.json	As 8 missões e a pontuação (a soma dos critérios precisa dar 100)
speakers.json	Instrutores, mentores, palestra e banca (name: null = card "a revelar")
prizes.json	Prêmios (reward: null = "a revelar")
partners.json	Realização e parceiros
faq.json	Perguntas frequentes
teams.json	Lista reserva de equipes (as oficiais vêm da planilha, ao vivo)

8. Inscrições (Google Sheets)

Fluxo: formulário → validação no navegador (regras do domínio) → `POST` para o Web App do Apps Script → nova linha na aba **Inscricoes** com protocolo `HR26-XXXXXX`. Para a equipe aparecer no site, a organização muda a coluna **Status** para `confirmado`. O site lê `?action=teams`, que expõe **só** nome da equipe, curso e quantidade de pessoas.

Passo a passo completo em `apps-script/README.md`. Depois de publicar o Web App, coloque a URL em `PUBLIC_REGISTRATION_ENDPOINT` (variável de ambiente na Cloudflare) ou em `registration.endpoint` no `event.json`.

Proteções: honeypot, tempo mínimo de preenchimento, validação no servidor, trava de concorrência (`LockService`), cache de 60s na lista de equipes, novas tentativas com backoff no cliente e consentimento LGPD.

9. Convenções de código

- Código, nomes de arquivos e commits em **inglês**. Textos da interface e comentários em **português**.
- Comentário curto e objetivo, só quando o código não se explica sozinho. Nada de comentar o óbvio.
- POO: classes pequenas com uma responsabilidade. **Arquivo com mais de 300 linhas gera aviso no lint**: quebre.
- Classes e componentes em `PascalCase`, funções e variáveis em `camelCase`, constantes globais em `UPPER_SNAKE_CASE`.
- Imports por alias de camada: `@domain/*`, `@application/*`, `@infrastructure/*`, `@presentation/*`, `@shared/*`, `@composition/*`.

- `import type` para tipos (obrigatório pelo `verbatimModuleSyntax`). TypeScript no modo mais estrito, sem `any`.
- CSS: use **só** os tokens de `styles/tokens.css`; estilos com escopo dentro do `.astro`; media queries `min-width` (mobile first).
- Acessibilidade e performance não são opcionais (checklist na seção 13).

10. Commits (Conventional Commits)

Formato: `<tipo>(<escopo>): <descrição curta no imperativo, em inglês>`

Tipo	Quando usar
feat	Nova funcionalidade
fix	Correção de bug
perf	Melhoria de performance
refactor	Mudança de código sem alterar comportamento
style	Formatação (sem mudar lógica)
test	Testes
docs	Documentação
build	Build e dependências
ci	Pipeline de CI
chore	Tarefas gerais
revert	Reverter commit

Escopos: `domain`, `application`, `infra`, `ui`, `sections`, `experience`, `motion`, `a11y`, `perf`, `seo`, `content`, `docs`, `ci`, `deps`, `config`, `tests`.

```
feat(sections): add prizes section with locked rewards
fix(experience): stop render loop when tab is hidden
content(faq): add question about team size        <- inválido: "content" é escopo, não tipo
docs(content): update schedule for day 14          <- correto
feat(domain)!: require team size rule              <- "!" = breaking change
```

O **commitlint** valida a mensagem no `commit-msg` e o **lint-staged** roda ESLint e Prettier no `pre-commit` (Husky). Mudanças em `package.json`, `docs/stack-manifest.json` ou neste guia regeneram os PDFs automaticamente.

11. Branches e Pull Requests

1. Atualize a `main` e crie a branch: `feat/<assunto>`, `fix/<assunto>`, `docs/<assunto>`.
2. Commits pequenos, no padrão acima.
3. `npm run validate` antes de abrir o PR.
4. Abra o PR pra `main` usando o template (checklist). A CI roda lint, testes e build; a Cloudflare gera uma URL de preview.
5. Pelo menos 1 revisão. Merge com **squash**, com título no padrão Conventional Commits.

Recomendado: proteger a `main` no GitHub (Settings !’ Branches), exigindo PR e CI verde.

12. Deploy na Cloudflare Pages (grátis)

1. Crie uma conta gratuita em `dash.cloudflare.com`.
2. **Workers & Pages !’ Create !’ Pages !’ Connect to Git** e escolha `Marcos-Codfy/lp-hackarun`.
3. Framework preset **Astro**. Build command `npm run build`. Output directory `dist`.
4. Variáveis de ambiente: `NODE_VERSION=24`, `PUBLIC_SITE_URL=https://<seu-projeto>.pages.dev`, `PUBLIC_REGISTRATION_ENDPOINT=<url do Web App>`.
5. **Save and Deploy**. Cada push na `main` publica em produção; cada PR ganha uma URL de preview.
6. (Opcional) Domínio próprio em **Custom domains**.

Cache e headers de segurança ficam em `public/_headers`.

Opção	Por que não foi a escolhida
Vercel (Hobby)	Termos do plano gratuito proíbem uso comercial/ patrocinado; banda limitada a 100 GB/mês
GitHub Pages	Banda sugerida de 100 GB/mês, sem headers customizados e sem preview por PR
Cloudflare Pages	Escolhida: banda e requisições ilimitadas pra estático, CDN global com servidores no Brasil, preview por PR, headers customizados

13. Checklist antes do merge

- [ ] `npm run validate` passou.
- [ ] Testei no celular (360px) e no desktop (1440px).
- [ ] Navegação só por teclado funciona; foco visível.
- [ ] Ativei "reduzir movimento" no sistema e a página continua utilizável.
- [ ] Sem texto com contraste baixo; imagens com `alt`.
- [ ] Nenhum arquivo passou de 300 linhas; nenhum `console.log` esquecido.
- [ ] Lighthouse mobile: Performance "e 90, Acessibilidade "e 95.

14. Versionamento

SemVer em `package.json` (`MAJOR.MINOR.PATCH`). Tags `vX.Y.Z` nos marcos (ex.: `v1.0.0` no lançamento das inscrições). As mensagens no padrão Conventional Commits permitem gerar changelog depois.

15. Pendências da organização

- [ ] URL do Web App de inscrições (`PUBLIC_REGISTRATION_ENDPOINT`).
- [ ] Prazo das inscrições (`registration.closesAt`) e capacidade de equipes (`registration.capacity`).
- [ ] Tamanho mínimo e máximo da equipe (`registration.teamSize`, hoje 2 a 5 provisório).
- [ ] Prêmios (`prizes.json`) e regulamento oficial (link).
- [ ] Nomes de instrutores, mentores, palestrantes e banca (`speakers.json`).
- [ ] Logos oficiais em SVG da Unicatólica e do Capim Dourado; logo HackaRun em camada separada (SVG ou PNG transparente).

- [ ] Áudio do globo final (`public/audio/cta-globe.mp3`, até 60 KB). Sem ele toca um som sintetizado.
- [ ] Imagem de compartilhamento 1200×630 (`public/og-image.jpg`).
- [ ] Local da caça aos NPCs (universidade ou shopping).
- [ ] Domínio próprio (opcional).

16. Documentação gerada

`npm run docs:pdf` gera em `docs/pdf/`:

- `HackaRun_LP_Stack_e_Padroes.pdf`: stack, bibliotecas (lidas do `package.json`), recursos, padrões e conceitos (só os nomes).
- `HackaRun_LP_Guia_do_Projeto.pdf`: este guia.

Se `DOCS_OUT_DIR` estiver definido no `.env`, os PDFs também são copiados pra essa pasta.